

ΧΡΙΣΤΟΠΟΥΛΟΣ ΚΩΝΣΤΑΝΤΙΝΟΣ ΑΜ:4527

ΣΚΑΡΑΦΙΓΚΑΣ ΒΑΣΙΛΕΙΟΣ ΑΜ:4491

ΠΡΩΤΟ ΣΕΤ ΑΣΚΗΣΕΩΝ:Classifiers

Πρώτη Μέθοδος: K-Κοντινότεροι Γείτονες

Αρχικά πριν την υλοποίηση της μεθόδου φορτώνω τα αρχεία 'csv' που σχετίζονται με το train και test που θα χρησιμοποιηθούν στα classifiers.

```
# ----- PREPARATIONS -----  
# open .csv files  
train_data = pd.read_csv("train.csv")  
test_data = pd.read_csv("test.csv")  
Y_data = pd.read_csv("sample_submission.csv")
```

Επιπλέον κάνω κάποιες μετατροπές στα υπάρχον δεδομένα(train,test) και συγκεκριμένα στο χαρακτηριστικό "color" των τεράτων όπου αντικαθιστώ κάθε ένα από τα έξι χρώματα με μια κανονικοποιημένη συνεχή τιμή στο διάστημα [0,1] όπως φαίνεται παρακάτω.

```
# change color data  
train_data["color"] = train_data["color"].replace(to_replace=['clear', 'white', 'green', 'blue', 'black', 'blood'],  
                                                  value=[1 / 6, 2 / 6, 3 / 6, 4 / 6, 5 / 6, 6 / 6])  
test_data["color"] = test_data["color"].replace(to_replace=['clear', 'white', 'green', 'blue', 'black', 'blood'],  
                                                value=[1 / 6, 2 / 6, 3 / 6, 4 / 6, 5 / 6, 6 / 6])
```

Κατόπιν εξάγω τα X,Y train and test sets που θα χρησιμοποιηθούν στα μοντέλα(δεν ασχολούμαι με τα id και δεν τα εκλαμβάνω υπόψιν στην διαδικασία). Η "Y_test" είναι ενδεικτική καθώς δεν έχω πραγματικά δεδομένα του πειράματος

```
# -----  
# Extracting Train and Test set  
X_train = train_data.iloc[:, [1, 2, 3, 4, 5]].values  
Y_train = train_data.iloc[:, 6].values  
X_test = test_data.iloc[:, [1, 2, 3, 4, 5]].values  
Y_test = Y_data.iloc[:, 1].values  
ID_test = Y_data.iloc[:, 0].values
```

ΠΑΡΑΤΗΡΗΣΗ:

Δημιουργώ με συνάρτηση για την εξαγωγή των αποτελεσμάτων των μεθόδων ώστε να τα ανεβάσουμε στο Kaggle για να εξετάσουμε το accuracy καθώς δεν έχω ενδεικτικά αποτελέσματα από τα πραγματικά δεδομένα του τεστ για να το εξετάσω απευθείας στο πρόγραμμα.

```
#Samples extraction
def sample_extraction(filename):
    with open(filename, 'w', newline='') as sample_submission_new:
        csvwriter = csv.writer(sample_submission_new) # 2. create a csvwriter object
        csvwriter.writerow(['id', 'type']) # 4. write the header
        csvwriter.writerows(results) # 5. write the rest of the data
```

Δημιουργώ μια συνάρτηση για την K-Neighbors η οποία υλοποιείται με Ευκλείδεια απόσταση (**metric='euclidean', weights='distance'**) . Αφού κάνω fit τα train data μου στον classifier κάνω predict τα X_test δεδομένα μου σε τι κατηγορία θα ανήκουν. Τέλος υπολογίζω τα accuracy, f1_score, precision και το recall. Υπενθυμίζουμε ότι τα νούμερα είναι ενδεικτικά στο πρόγραμμα καθώς μπορούμε να εξετάσουμε μόνο το accuracy μέσω της ιστοσελίδας Kaggle. Τέλος εξετάζω για K=1,3,5,10 γείτονες και εξάγω τα samples για να τα εξετάσω στο Kaggle.

```
# K NEAREST NEIGHBORS
def KNeighborsClassifierEvaluation():
    # Fitting K-NN classifier to the training set
    K=[1,3,5,10] #values for Neighbors test
    for i in range(len(K)):
        print("KNN Method for K= ",K[i])
        classifier = KNeighborsClassifier(n_neighbors=K[i], weights='distance')
        classifier.fit(X_train, Y_train)

        # Predicting the test set result
        y_pred = classifier.predict(X_test)

        # Can't verify the results.....
        accuracy = accuracy_score(Y_test, y_pred)
        F1_score = f1_score(Y_test, y_pred, average='weighted')
        Recall_score = recall_score(Y_test, y_pred, average='weighted')
        Precision_score = precision_score(Y_test, y_pred, average='weighted')
        print(f"K = {K} : Accuracy: {accuracy}, f1 score: {F1_score}, recall_score: {Recall_score}, precision_score: {Precision_score}")

        # Sample extraction file
        results = [list(x) for x in zip(ID_test, y_pred)]
        filename = 'sample_submission_KNN'+str(K[i])+'.csv'
        sample_extraction(filename, results)

    return 0

KNeighborsClassifierEvaluation()
```

Έγινε χρήση των βιβλιοθηκών "sklearn" για το KNN , csv και panda για την εργασία μας πάνω στα αρχεία.

```
# Data Manipulation
import pandas as pd #for csv files
import csv
import os

#for K-Neighbors
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import f1_score, accuracy_score, recall_score, precision_score
```

Ακολουθεί ο πίνακας Accuracy:

Απόσταση	K	Accuracy Score
Ευκλείδεια	1	0.67485
	3	0.66918
	5	0.68620
	10	0.72211

Δεύτερη Μέθοδος: Νευρωνικά Δίκτυα

Τα νευρωνικά δίκτυα είναι μια μέθοδος που σχετίζεται με τον τρόπο μάθησης του ανθρώπινου εγκεφάλου και έχει αναπτυχθεί τις τελευταίες δεκαετίες. Αποτελείται από τους λεγόμενους νευρώνες οι οποίοι είναι υπεύθυνοι για την μάθηση. Συγκεκριμένα έχουμε διάφορα επίπεδα από τα οποία αποτελείται ένα τέτοιο δίκτυο. Ένα επίπεδο εισόδου, ένα ή περισσότερα κρυμμένα επίπεδα και ένα επίπεδο εξόδου. Το νευρωνικό δίκτυο αποτελείται από βάρη που παίρνουν μια τιμή και σχετίζεται με την “γνώση” του νευρωνικού συστήματος και οι τιμές τους μεταβάλλονται κατά την διάρκεια της εκπαίδευσης. Στην συγκεκριμένη άσκηση θα δημιουργήσουμε δύο μοντέλα

- Με ένα κρυμμένο επίπεδο και K(50,100,200) κρυμμένους νευρώνες
- Με δύο κρυμμένα επίπεδα και K1,K2 κρυμμένους νευρώνες
 - $(K1,K2) = (50,25),(100,50),(200,100)$

Το επίπεδο της εξόδου θα αποτελείται από 3 νευρώνες(όσες είναι και οι κατηγορίες των δεδομένων) με συνάρτηση ενεργοποίησης softmax που θα μας επιστρέφει πιθανότητα σε ποια κατηγορία θα ανήκει το κάθε τέρας. Επιπλέον για την εκπαίδευση θα χρησιμοποιήσουμε την μέθοδο βελτιστοποίησης “Stochastic Gradient Descent”.

Αρχικά αλλάζω τα type του train data από string σε νούμερα έτσι ώστε κάθε τύπος φαντάσματος να αντιστοιχεί σε έναν αριθμό (0,1,2). Στην συνέχεια σιγουρεύομαι ότι όλα τα δεδομένα που θα βάλω στην μηχανή μου είναι

τύπου float32 με την βοήθεια της numpy τα μετατρέπω σε numpy.array στα csv δεδομένα και στην συνέχεια χωρίζω τα train και test data που θα χρειαστεί να κάνω fit στο μοντέλο μου για να το εκπαιδεύσω.

```
def Neural_Net(train_data, test_data, ID_test):
    #edit data before we start
    train_data['type'] = LabelEncoder().fit_transform(train_data['type'])

    #float convert
    train_data = train_data.astype('float32')
    test_data = test_data.astype('float32')

    #numpy data
    train_data = train_data.to_numpy()
    test_data = test_data.to_numpy()

    #new data
    X_train = train_data[:, [1, 2, 3, 4, 5]]
    Y_train = train_data[:, 6]
    X_test = test_data[:, [1, 2, 3, 4, 5]]
```

Κατόπιν φτιάχνω 2 λίστες με τους K1,K2 νευρώνες που θα χρειαστώ αντίστοιχα για κάθε μοντέλο και κάνω fix το μοντέλο μου όπου στο επίπεδο εισόδου παίρνει πέντε διαφορετικά δεδομένα, μετά φτιάχνω τα κρυμμένα επίπεδα (layers) με τους κατάλληλους νευρώνες (dense) και ορίζω μέθοδο βελτιστοποίησης την "softmax".

```
K1 = [50, 100, 200]
K2 = [25, 50, 100]
# FIRST MODEL WITH ONE HIDDEN LAYER
# fix model
for i in range(len(K1)):
    model = Sequential()
    model.add(Input(shape=(5,))) # the inputs
    model.add(Dense(K1[i], activation='sigmoid')) # denses K
    # model.add(Dropout(.1))
    model.add(Dense(3, activation='softmax')) # activation format
    execute_model(model, K1[i], 0)
# SECOND MODEL WITH TWO HIDDEN LAYERS
for i in range(len(K1)):
    model = Sequential()
    model.add(Input(shape=(5,))) # the inputs
    model.add(Dense(K1[i], activation='sigmoid')) # denses K
    model.add(Dense(K2[i], activation='sigmoid')) # denses K
    # model.add(Dropout(.1))
    model.add(Dense(3, activation='softmax')) # activation format
    execute_model(model, K1[i], K2[i])
return 0

Neural_Net(train_data, test_data, ID_test)
```

Τέλος καλώ την συνάρτηση << execute_model>> όπου κάνω compile το μοντέλο μου και παράγω τα τελικά αρχεία με τα samples.

```
def execute_model(model,K1,K2):
    # compiler and fit
    model.compile(optimizer='sgd', loss='sparse_categorical_crossentropy',metrics=['sparse_categorical_accuracy']) # optimize met
    model.fit(X_train, Y_train, epochs=50, verbose=0) # epochs and fit data
    y_pred = model.predict(X_test) # prediction

    # sample extraction
    y_pred = np.array(y_pred) # np.array
    indices = np.argmax(y_pred, axis=1) # max num index of every row
    indices = indices.tolist() # convert to list

    # replace numbers with type category
    for i in range(len(indices)):
        if indices[i] == 0:
            indices[i] = 'Ghoul'
        elif indices[i] == 1:
            indices[i] = 'Goblin'
        else:
            indices[i] = 'Ghost'
    # extract csv file
    results = [list(x) for x in zip(ID_test, indices)]
    if(K2==0):
        filename = 'sample_submission_Neural_Network-' + str(K1) + '.csv'
        sample_extraction(filename, results)
    else:
        filename = 'sample_submission_Neural_Network-(' + str(K1) + '-' + str(K2) + ').csv'
        sample_extraction(filename, results)
```

Κάνω κάποιες μετατροπές στην y_pred ώστε να μετατρέψω τα ποσοστά ξανά στον σωστό τύπο φαντάσματος πριν τα εισάγω στο csv αρχείο μου για να το ανεβάσω στο Kaggle. Η λίστα "Indices" περιλαμβάνει τον κάθε νικητή νευρώνα που αντιστοιχεί σε ένα τύπο με βάσει το ποσοστό που προέκυψε στην y_pred και την μετατρέπω όπως ανέφερα παραπάνω στον κατάλληλο τύπο φαντάσματος.

Ακολουθεί ο πίνακας με τα Accuracy:

K	Accuracy Score
50	0.71345
100	0.71430
200	0.71921
(50,25)	0.71475
(100,50)	0.71518
(200,100)	0.72121

Οι βιβλιοθήκες που χρησιμοποιήθηκαν είναι οι εξής:

```
# for MLP
import numpy as np
# import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout
from sklearn.preprocessing import LabelEncoder
```

Τρίτη Μέθοδος: Μηχανές Διανυσματικής Υποστήριξης (SVM)

Οι Μηχανές Διανυσματικής Υποστήριξης μεταφέρουν δεδομένα που είναι γραμμικά διαχωρισμένα στον “χώρο χαρακτηριστικών” βασισμένο στην μέθοδο ταξινόμησης σε πυρήνες (kernels). Οι συναρτήσεις των πυρήνων δίνουν το εσωτερικό γινόμενο στο χώρο χαρακτηριστικών εκτελώντας υπολογισμούς στο χώρο δεδομένων. Εμείς θα υλοποιήσουμε

- Γραμμική συνάρτηση πυρήνα και
- Gaussian συνάρτηση πυρήνα

Επειδή οι μηχανές αυτές υποστηρίζουν κατηγοριοποίηση 2 τάξεων (binary) , θα χρησιμοποιήσουμε την τεχνική One-vs-Rest ώστε να κάνουμε κατηγοριοποίηση περισσότερων τάξεων όπως μας ζητείτε. Αναφορικά να σημειωθεί ότι κάθε κλάση-κατηγορία που έχουμε θα συγκριθεί με όλες τις άλλες. Στην περίπτωση αυτή για να μειώσουμε την πολυπλοκότητα και τον χρόνο εκτέλεσης μπορούμε να κάνουμε μείωση διάστασης (pca), μείωση του misclassification enalty (C) , εκπαίδευση σε υποσύνολα. Στην συγκεκριμένη άσκηση χρησιμοποιήθηκε η misclassification enalty (C) μόνο στην linear

συνάρτηση με $C=10$.

```
# ----- SUPPORT VECTOR MACHINES -----
def SVM_Kernels():
    # create classifier for OneVsRest
    print("\nSVM Methods (linear-RBF) with OVA implementation:")

    # 1.linear
    cls = OneVsRestClassifier(SVC(kernel='linear', class_weight='balanced', C=10))
    # 2.Gaussian rbf
    cls2 = OneVsRestClassifier(SVC(kernel='rbf', class_weight='balanced'))

    # train models
    cls.fit(X_train, Y_train)
    cls2.fit(X_train, Y_train)

    # predict response
    y_pred = cls.predict(X_test)
    pred = cls2.predict(X_test)

    # results for linear
    accuracy = accuracy_score(Y_test, y_pred)
    F1_score = f1_score(Y_test, y_pred, average='weighted')
    Recall_score = recall_score(Y_test, y_pred, average='weighted')
    Precision_score = precision_score(Y_test, y_pred, average='weighted')
    print(f"Linear SVM : Accuracy: {accuracy}, f1 score: {F1_score}, recall_score: {Recall_score}, precision_score: {precision_score}")

    # results for RBF
    accuracy = accuracy_score(Y_test, y_pred=pred)
    F1_score = f1_score(Y_test, y_pred=pred, average='weighted')
    Recall_score = recall_score(Y_test, y_pred=pred, average='weighted')
    Precision_score = precision_score(Y_test, y_pred=pred, average='weighted')
    print(f"RBF SVM : Accuracy: {accuracy}, f1 score: {F1_score}, recall_score: {Recall_score}, precision_score: {precision_score}")

    # Sample extraction files
    results = [list(x) for x in zip(ID_test, y_pred)]
    results2 = [list(x) for x in zip(ID_test, pred)]

    # for linear
    filename = 'sample_submission_SVM_linear.csv'
    sample_extraction(filename, results)
    # for RBF
    filename = 'sample_submission_SVM_RBF.csv'
    sample_extraction(filename, results2)

    return 0

SVM_Kernels()
```

Συγκεκριμένα όπως φαίνεται στην παραπάνω εικόνα φτιάχνω δύο classifiers ένα με kernel linear και ένα με kernel gaussian rbf όπως ζητείται από την άσκηση. Στην συνέχεια κάνω αντίστοιχα fit τα δεδομένα μου για το train του συστήματος και τέλος κάνω το prediction για κάθε kernel ξεχωριστά. Κατόπιν υπολογίζω το accuracy, recall, precision, f1_score τα οποία όπως προαναφέραμε είναι καθαρά ενδεικτικά. Τέλος εξάγω τα samples για κάθε ένα kernel και τα εξετάζω στο Kaggle.

Ακολουθεί ο πίνακας Accuracy:

Kernel	Accuracy Score
Linear	0.73156
Gaussian	0.72400

Οι βιβλιοθήκες που χρησιμοποιήσαμε είναι οι εξής:

```
#for K-Neighbors and SVM
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import f1_score, accuracy_score, recall_score, precision_score
```

```
# for SVM
from sklearn import svm
from sklearn.svm import SVC
from sklearn.multiclass import OneVsRestClassifier
```

Τέταρτη Μέθοδος: Naïve Bayes Classifier

$$P(y_j|X) = \frac{P(X|y_j)P(y_j)}{P(X)}$$

Παραπάνω παραθέτουμε τον κανόνα του Bayes, όπου

- X : δεδομένα/χαρακτηριστικά του αρχείου train.csv(bone_length..),
- Yj : αντιστοιχεί στις 3 κατηγορίες τεράτων που πρέπει να κατηγοριοποιήσουμε(j = {1,2,3}).
- Ο όρος : P(X|Yj) γράφεται ως :
 $P(X|W_j) = P(X_1|W_j) * P(X_2|W_j) * P(X_3|W_j) * P(X_4|W_j) * P(X_5|W_j)$
 Όπου για τα 4 πρώτα features (X1 έως X4) χρησιμοποιούμε γκαουσιανη κατανομή και για το τελευταίο(X5) χαρακτηριστικό χρησιμοποιούμε πολυωνυμική.

$$p(x_i|y_j) = \frac{1}{\sqrt{2\pi\sigma_j^2}} e^{-\frac{(x_i-\mu_j)^2}{2\sigma_j^2}}$$

- Η μέση τιμή και η τυπική απόκλιση υπολογίζονται από το δοθέν train.csv αρχείο.

$$\mu_{j1} = (\sum X_{i1})/n_j$$

$$\sigma_{j1}^2 = (\sum (X_{i1}-\mu_{j1})^2)/n_j-1$$